UTN
FACULTAD REGIONAL
RAFAELA



Unidad 2: HTML

Programación 3

UTN FRRa 2026

Docente: Lucas Ferrero

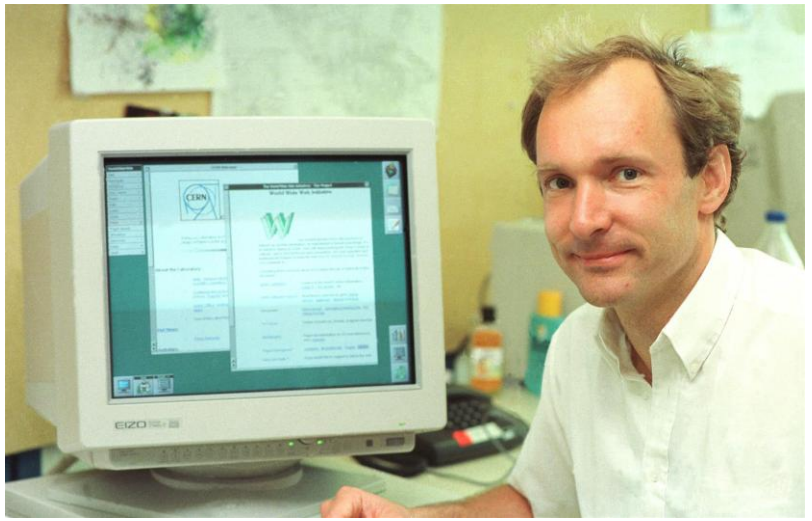
Historia y Evolución de HTML

World Wide Web

The WorldWideWeb (W3) is a wide-area hypermedia information retrieval initiative aiming to give universal access to a large universe of documents.

Everything there is online about W3 is linked directly or indirectly to this document, including an executive summary of the project, Mailing lists , Policy , November's W3 news , Frequently Asked Questions .

What's out there?
Pointers to the world's online information, subjects , W3 servers, etc.
Help
on the browser you are using
Software Products
A list of W3 project components and their current state. (e.g. Line Mode ,X11 Viola , NeXTStep , Servers , Tools , Mail robot , Library)
Technical
Details of protocols, formats, program internals etc
Bibliography
Paper documentation on W3 and references.
People
A list of some people involved in the project.
History
A summary of the history of the project.
How can I help ?
If you would like to support the web..
Getting code
Getting the code by anonymous FTP , etc.



Los Orígenes del Lenguaje

HTML nació en 1991 de la mano de Tim Berners-Lee, un físico del CERN en Suiza, quien buscaba una manera de compartir documentos científicos entre investigadores de diferentes instituciones. Lo que Berners-Lee no imaginaba era que estaba creando el lenguaje fundamental que permitiría el desarrollo de la World Wide Web tal como la conocemos hoy.

La primera versión de HTML contenía apenas 18 elementos, muchos de los cuales aún utilizamos: títulos, párrafos, enlaces e imágenes. Era un lenguaje extremadamente simple, diseñado con un único propósito en mente: estructurar documentos académicos y permitir la navegación entre ellos mediante hipervínculos.

La Evolución a Través de las Versiones

- **HTML 2.0** llegó en 1995 y estableció el primer estándar formal. Fue la base sobre la cual se construyó la primera generación de sitios web comerciales. En esta etapa, HTML comenzó a incluir elementos para formularios, lo que permitió la interacción bidireccional entre usuarios y servidores, transformando la web de un medio puramente informativo a uno interactivo.
- **HTML 3.2**, lanzado en 1997, introdujo tablas, applets y text flow alrededor de imágenes. Esta versión marcó el inicio de la "guerra de navegadores" entre Netscape e Internet Explorer, donde cada navegador implementaba sus propias extensiones propietarias de HTML, creando problemas de compatibilidad.

- **HTML 4.01**, publicado en 1999, representó un punto de inflexión importante. Esta versión introdujo hojas de estilo CSS de manera formal, separando el contenido de la presentación. También mejoró la accesibilidad y el soporte para internacionalización. HTML 4.01 fue tan robusto que permaneció como estándar durante más de una década.

El Intento Fallido: XHTML

A principios de los años 2000, el W3C intentó reimaginar HTML como una aplicación de XML, creando XHTML. La idea era llevar el rigor sintáctico de XML al desarrollo web: documentos perfectamente formados, validación estricta, y una sintaxis más predecible. Sin embargo, XHTML resultó demasiado estricto para el ecosistema web real. Un simple error de sintaxis podía romper completamente una página, y muchos desarrolladores lo encontraron poco práctico para proyectos del mundo real.

HTML5: El Renacimiento

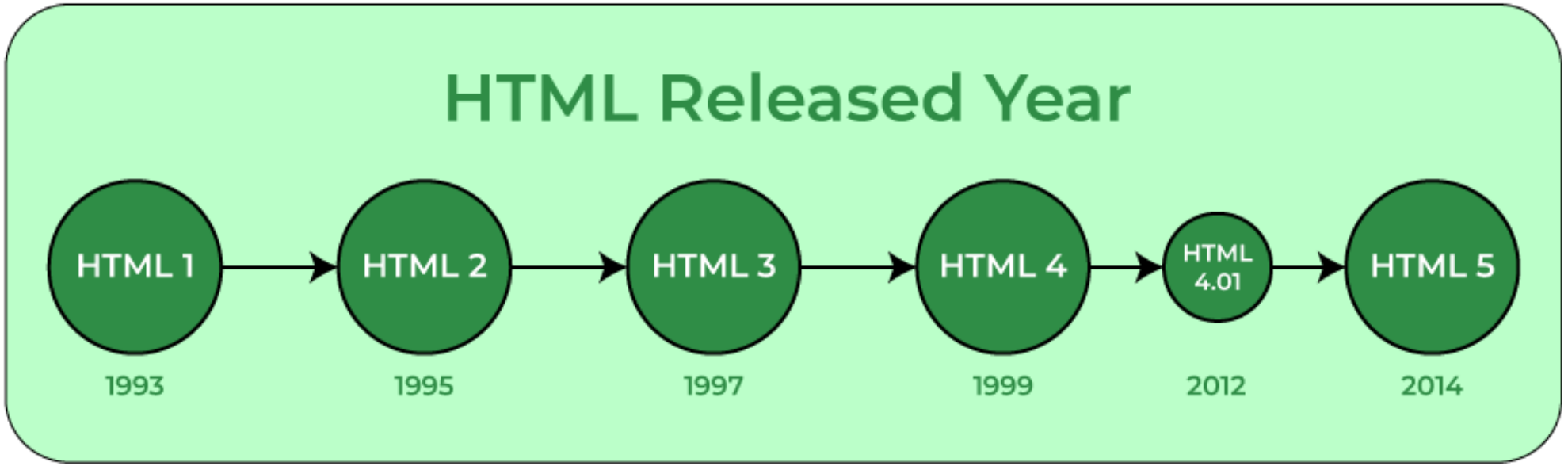
En 2004, un grupo de desarrolladores de Apple, Mozilla y Opera formaron el WHATWG (Web Hypertext Application Technology Working Group) porque consideraban que la W3C estaba perdiendo el rumbo con XHTML. Comenzaron a trabajar en lo que eventualmente se convertiría en HTML5.

HTML5, oficialmente publicado en 2014, fue revolucionario. No solo agregó nuevos elementos semánticos y APIs, sino que cambió fundamentalmente la filosofía del lenguaje. HTML5 abraza la realidad del desarrollo web: es permisivo con los errores, prioriza la compatibilidad hacia atrás, y está diseñado para soportar aplicaciones web complejas, no solo documentos estáticos.

Esta versión introdujo elementos semánticos como <header>, <nav>, <article>, <section>, y <footer>, que permiten crear documentos con estructura significativa. También incorporó APIs nativas para video, audio, canvas para gráficos, geolocalización, almacenamiento local, y mucho más. HTML5 convirtió al navegador en una plataforma de aplicaciones completa.

El Estándar Viviente

Desde 2019, HTML ya no tiene números de versión. Es ahora un "living standard" (estándar viviente) mantenido por el WHATWG. Esto significa que HTML evoluciona continuamente, con pequeñas mejoras y nuevas características añadiéndose regularmente sin esperar grandes lanzamientos. Este modelo refleja mejor la naturaleza dinámica de la web moderna.



Anatomía de un elemento HTML

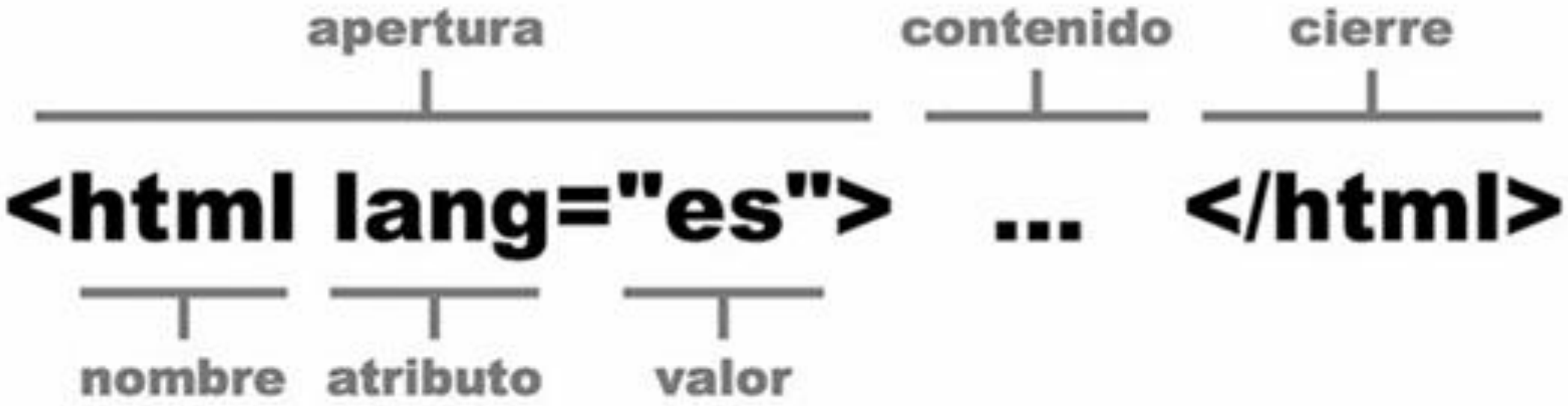


Figura 1-4: *Elemento HTML*

- Se deben combinar múltiples elementos para definir un documento. Los elementos son listados en secuencia de arriba abajo y pueden contener otros elementos en su interior.
- Existen elementos que no poseen etiqueta de cierre. A estos elementos se les suelen llamar “elementos o etiquetas individuales”. Algunos ejemplos son: <img />, <input/>, <meta />,
- **HTML es un lenguaje de marcado basado en etiquetas**

Atributos

- Los atributos ofrecen información acerca del elemento y su contenido.
- Los atributos poseen un nombre y valor. Ejemplo: lang=“es”

Estructura

- Definiremos los elementos esenciales
- Elementos estructurales:
  - <html>
  - <head>: cabecera
  - <body>: cuerpo

<head>

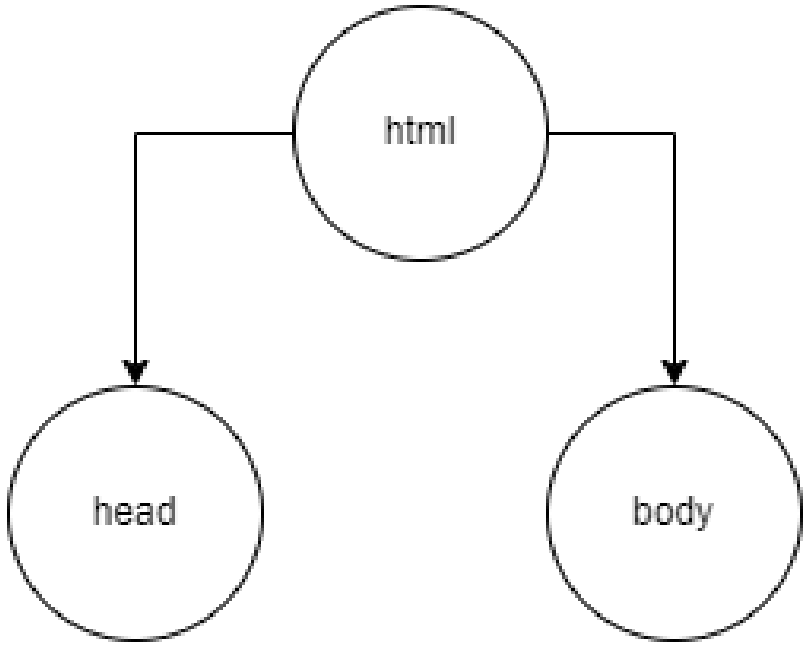
- El contenido de la etiqueta no es visible para el usuario
- Incluye el título de la página web
- Se especifica el código de caracteres: UTF-8
- Incorpora archivos externos con estilos o códigos necesarios para generar la página

<body>

- El cuerpo es la parte visible del documento que se especifica en el elemento <body>

Estructura de árbol

- La estructura HTML puede describirse como un árbol
- Cada elemento en el árbol posee una jerarquía



Elementos del <head>

- **<title>** → Define el título de la página web.
- **<meta>** → Representa metadatos asociados al documento. Dispone de los atributos *name, content y charset*
- **<link>** → Se utiliza para cargar recursos externos como archivos CSS y fuentes de texto.
- **<style>** → Se usa para declarar código CSS dentro del documento HTML.
- **<script>** → Este elemento se usa para cargar o declarar código Javascript

Elementos estructurales en el <body>

- ** ** → Define una división genérica
- **<main>** → División que contiene el contenido principal del documento HTML (el tema central de la página)
- **<nav>** → Elemento para definir navegación en el sitio.
- **<section>** → Define una sección genérica
- **<aside>** → Elemento que define información adicional al contenido principal
- **<article>** → Representa un artículo independiente

- **<header>** → Elemento para definir cabeceras
- **<footer>** → Elemento para definir pies, como por ejemplo, el pie de página.



Ver listado 2-14

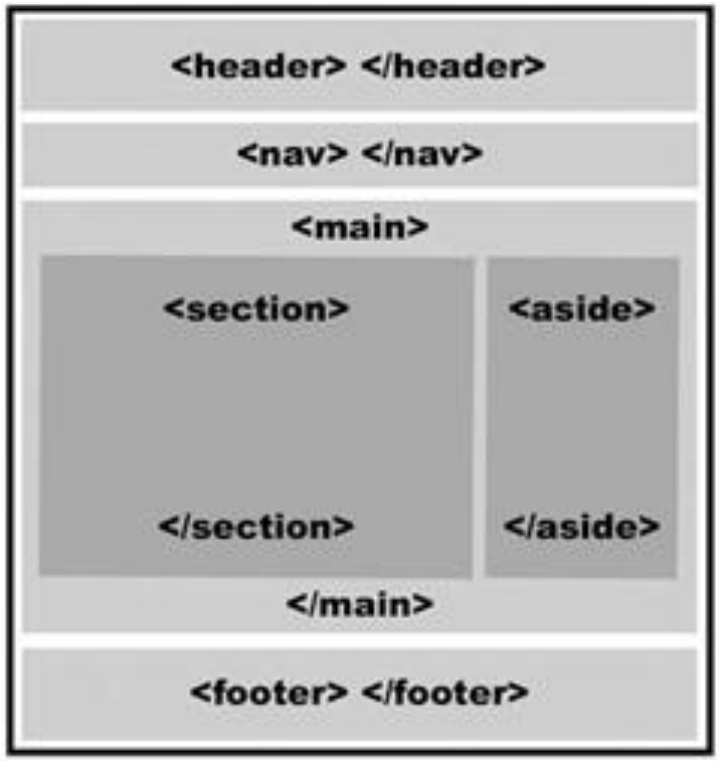
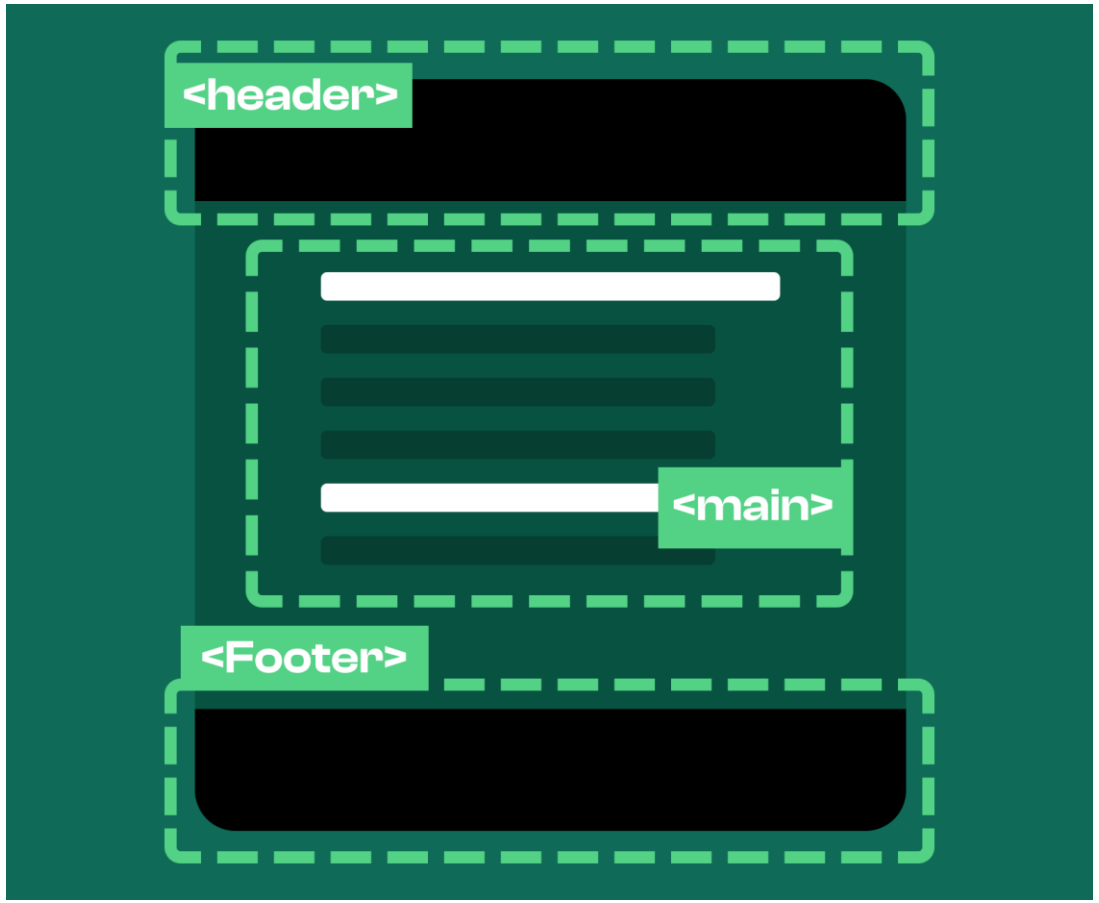
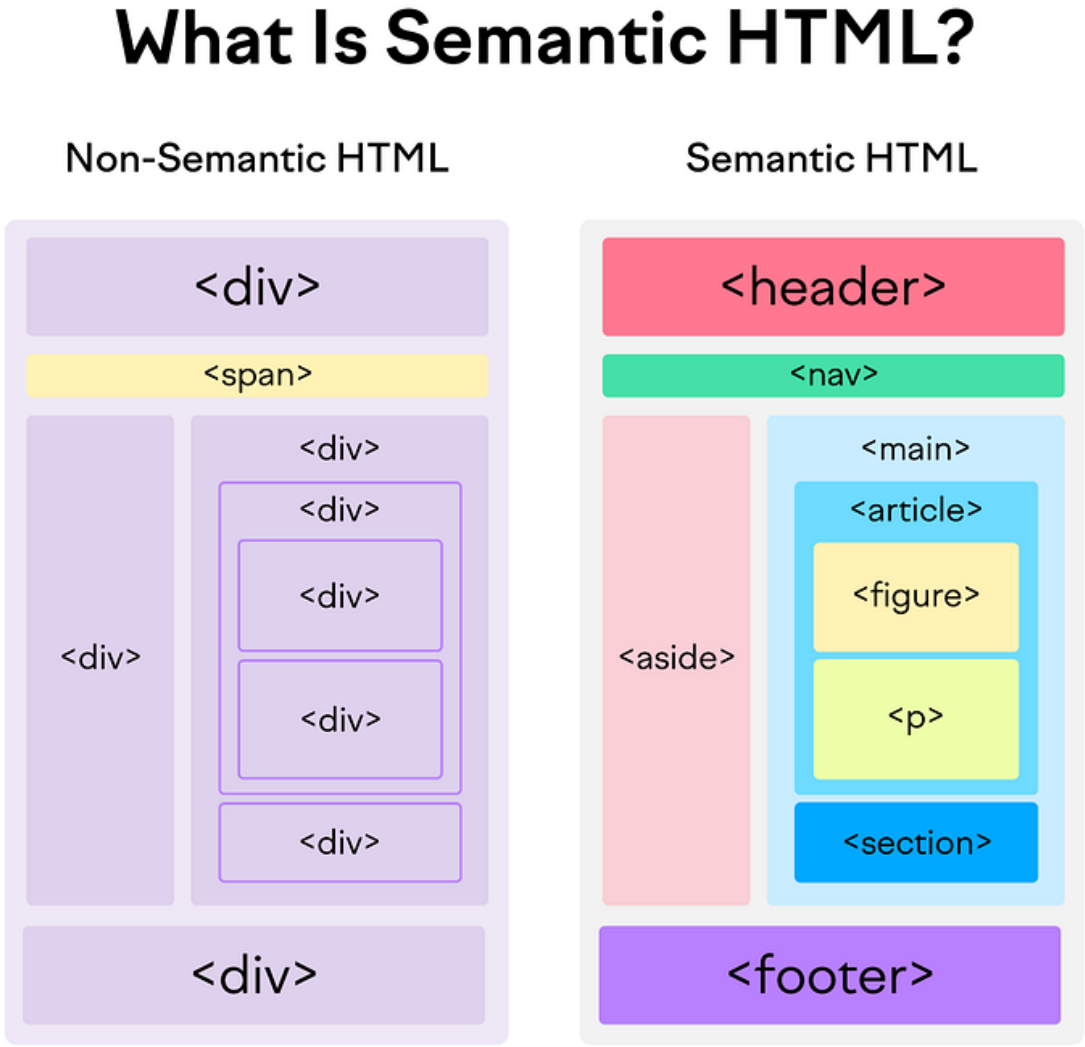


Figura 2-2: *Representación visual de un diseño web tradicional*

Figura 2-3: *Representación de la estructura de un documento usando elementos de HTML5*

Estructura Semántica





La Importancia de la Semántica en HTML5

La semántica en HTML no es solo sobre elegancia del código; tiene implicaciones prácticas profundas. Cuando utilizamos <article> en lugar de , **estamos comunicando significado**. Los navegadores, motores de búsqueda y tecnologías asistivas pueden entender ese significado y actuar en consecuencia.

Considera la diferencia entre estos dos fragmentos de código. El primero utiliza elementos genéricos sin significado semántico, mientras que el segundo comunica estructura y propósito claramente. Un lector de pantalla puede saltar directamente a la navegación principal cuando el usuario lo solicite si usamos <nav>. No puede hacer lo mismo con un .

Elementos Semánticos HTML5

El elemento <*main*> debe usarse una sola vez por página y contiene el contenido principal del documento.

<*article*> representa contenido independiente que podría distribuirse de forma autónoma. Un post de blog, un comentario de usuario, o un widget interactivo son ejemplos perfectos. Si puedes extraer ese bloque de HTML y tiene sentido por sí mismo, probablemente debería ser un <article>.

<*section*> es para agrupar contenido temáticamente relacionado. La regla general es que cada <*section*> debería tener un encabezado. Si estás usando <*section*> solo para estilizar, probablemente deberías usar .

<*aside*> representa contenido relacionado (o adicional) del contenido principal. Barras laterales, o información complementaria son usos apropiados. No uses <*aside*> simplemente porque algo está visualmente al lado del contenido principal.

<*header*> y <*footer*> pueden usarse múltiples veces en una página. Puedes tener un <*header*> para la página, pero también un <*header*> dentro de cada <*article*>. Lo mismo aplica para <*footer*>.

Atributos globales

- Los atributos globales son atributos comunes a todos los elementos HTML; se pueden usar en todos los elementos
- **id** → Se utiliza para asignar un identificador único a un elemento
- **class** → Se usa para asignar uno o varios nombres de clases a un elemento

- El atributo *id* identifica elementos independientes con un valor único, mientras que el valor del atributo *class* se puede duplicar para asociar elementos con características similares.

Contenido - Texto

- **<h1>** → Este elemento representa un título. Hay hasta 6 niveles (<h2>, <h3>, <h4>, <h5>, <h6>).
- **<p>** → Este elemento representa un párrafo.
- **** → Puede contener un párrafo, una frase o una palabra. Se utiliza para asignar estilos personalizados.
- **
** → Se utiliza para introducir saltos de línea
- **** → Indica importancia
- **** → Este elemento se usa para indicar énfasis

Contenido - Enlaces

Conectar documentos con otros documentos mediante enlaces es lo que hace posible la Web.

- **<a>** → Este elemento crea un enlace. Incluye el atributo *href* para especificar la URL del enlace.
- Para desplazarnos a un elemento dentro del documento actual utilizar #id en *href*
- Los enlaces se pueden usar para conectar el documento actual con otros documentos del mismo sitio o en otros sitios

- El elemento <a> también se puede usar para crear enlaces a aplicaciones. HTML ofrece las palabras clave *mailto* y *tel* para especificar una cuenta de correo o un número de teléfono.

Ejemplo: href=“mailto:info@utn.org.ar”

Contenido - Imágenes

Las imágenes pueden considerarse el segundo medio más importante en la Web.

- **<img>** → Este elemento inserta una imagen en el documento. Requiere el atributo *src* que especifica la URL de la imagen.

Atributos

- **src** → URL de la imagen
- **width** → Declara el ancho de la imagen.
- **height** → Declara la altura de la imagen.
- **alt** → Texto para mostrar si la imagen no se pudo cargar.

Contenido - Listas

A menudo, la información se debe representar como una lista de ítems.

- **<ul>** (Lista no ordenada) → Crea una lista de ítems sin orden.
- **<ol>** (Lista ordenada) → Crea una lista de ítems con orden.

Ambos elementos trabajan en conjunto con el elemento <li> para definir cada uno de los ítems de la lista.

Formularios

Los formularios son herramientas que podemos incluir en un documento para permitir a los usuarios insertar información. Esta información luego es enviada al servidor.

- **<form>** → Elemento que define un formulario.

Elemento de formularios

- **<input>** → Elemento que crea un campo de entrada. Puede recibir diferentes tipos de entradas dependiendo de su atributo *type*
- **<textarea>** → Elemento que genera campo de entrada para insertar textos extensos
- **<select>** → Elemento que crea una lista de opciones que el usuario puede elegir. Trabaja en conjunto con el elemento <option> para definir cada opción.
- **<button>** → Genera un botón. Incluye el atributo *type* para definir el propósito del botón.
- **<label>** → Elemento que crea una etiqueta para identificar a un elemento del formulario.

El elemento <*input*> es el más versátil de todos. Puede adoptar diferentes características y aceptas varios tipos de valores dependiendo del valor de su atributo *type.*

Valores disponibles para el atributo *type:*

- ***text*** → Genera un campo de entrada para insertar texto genérico
- ***email*** → Genera un campo de entrada para insertar cuentas de correos
- ***tel*** → *Genera un campo de entrada para insertar números de teléfonos*
- ***number*** → Genera un campo de entrada para insertar números
- ***date*** → *Genera un campo de entrada para insertar una fecha*
- ***password*** →*Genera un campo de entrada para insertar una constraseña*
- ***checkbox*** → *Genera una casilla de control que permite al usuario activarla o desactivarla*
- ***radio*** → Este valor genera un botón de opción. El usuario selecciona una opción de las posibles

- El atributo *value* se puede utilizar para declarar el valor inicial de un control

```html
<html lang="es">
  <head>
    <meta charset="utf-8">
    <title>Formularios</title>
  </head>
  <body>
    <section>
      <form>
        <div><input type="text" name="nombre"></div>
        <div><input type="text" name="edad" value="29"></div>
        <div><button type="submit">Enviar datos</button>
      </form>
    </section>
  </body>
</html>

```

<label>

- Los formularios necesitan incluir descripciones que le indiquen al usuario los datos que debe introducir. Por esta razón, HTML incluye el elemento <label>. Debido a que estos elementos no solo le indican al usuario qué valor debe introducir, sino que además ayudan al navegador a identificar cada parte del formulario, tienen que asociarse al elemento al que están describiendo.
- La manera más simple de asociar un <label> a un control es la siguiente:

```
1 <label>Tu apellido <input type="text" id="Apellido" name="apellido" /></label>
2
```

- Otra alternativa para asociar un elemento *<label>* con su elemento de formulario es implementando el atributo *for*. El atributo *for* conecta el elemento *<label>* con el elemento de formulario por medio del valor del atributo *id.*

```html
<section>
  <form>
    <div>
      <label for="nombre">Nombre: </label>
      <input type="text" name="nombre" id="nombre">
    </div>
    <div>
      <label for="edad">Edad: </label>
      <input type="text" name="edad" id="edad">
    </div>
    <div>
      <input type="submit" value="Enviar">
    </div>
  </form>
</section>
```

Validación HTML5 Nativa

HTML5 introdujo capacidades de validación nativas que permiten crear formularios robustos sin JavaScript. El atributo *required* marca campos obligatorios. El atributo *pattern* acepta expresiones regulares para validación personalizada. Los tipos de *input* especializados como *email* y *url* validan automáticamente el formato.

Sin embargo, como desarrolladores, debemos entender las limitaciones. La validación del lado del cliente es conveniente para el usuario, pero nunca debe ser tu única línea de defensa. **Los datos deben validarse siempre en el servidor, ya que la validación del cliente puede ser fácilmente salteada**.

Atributos de los distintos tipos de elementos de formulario

Cada tipo de elemento tiene la posibilidad de determinar atributos que modifican su comportamiento o establecen reglas de control.

Para type=“text”

- **maxlength** → Especifica el máximo número de caracteres que se permite introducir en el campo.
- **minlength** → Especifica el mínimo número de caracteres que se permite introducir en el campo.

- Para *type=“number”*
- **min** → Valor mínimo que acepta el campo
- **max** → Valor máximo que acepta el campo
- **step** → Es el número por el cual el campo se incrementa o reduce

```
1 <div>
2   <label>Nombre: <input type="text" name="nombre" maxlength="15"></label>
3 </div>
4
```

```
1 <input type="number" name="edad" min="13" max="100">
```

<input type=“radio”>

- Este elemento nos permite crear una lista de opciones que el usuario puede seleccionar con solo hacer clic en el botón correspondiente.
- Los elementos *<input>* se asocian entre ellos por medio del valor del atributo *name*, y el valor de cada opción se define por el atributo *value*
- El usuario solo puede seleccionar una opción a la vez
- El valor asignado al atributo *value* del elemento seleccionado es el que se enviará al servidor.

```html
<form name="formulario" method="get" action="procesar.php">
  <div>
    <label>Nombre: <input type="text" name="nombre" maxlength="15"></label>
  </div>
  <div>
    <label><input type="radio" name="edad" value="20" checked> 20 Años</label>
  </div>
  <div>
    <label><input type="radio" name="edad" value="30"> 30 Años</label>
  </div>
  <div>
    <label><input type="radio" name="edad" value="45"> 45 Años</label>
  </div>
  <div>
    <label><input type="radio" name="edad" value="60"> 60 Años</label>
  </div>
  <div>
    <input type="submit" value="Enviar">
  </div>
</form>
```

<input type=“checkbox”>

- El usuario puede seleccionar múltiples valores haciendo clic en las casillas correspondientes.
- El tipo *checkbox* es similar al tipo radio, pero tenemos que asignar diferentes nombres a cada elemento porque el usuario puede seleccionar varias opciones al mismo tiempo.
- Cuando el usuario selecciona una o más opciones, los valores de todos esos elementos se envían al servidor
- El atributo *checked* selecciona la opción al inicio

```
<form name="formulario" method="get" action="procesar.php">
  <div>
    <label>Nombre: <input type="text" name="nombre" maxlength="15"></label>
  </div>
  <div>
    <label>
      <input type="checkbox" name="edad15" value="20" checked>20 Años
    </label>
  </div>
  <div>
    <label>
     <input type="checkbox" name="edad30" value="30" checked> 30 Años
    </label>
  </div>
  <div>
    <label><input type="checkbox" name="edad45" value="45">45 Años</label>
  </div>
  <div>
    <label><input type="checkbox" name="edad60" value="60">60 Años</label>
  </div>
  <div><input type="submit" value="Enviar"></div>
</form>
```

Campos de fechas

- <input type="date”>
- <input type=“datetime-local”>
- <input type=“time”>

```
1 <form name="formulario" method="get" action="procesar.ph
2   <input type="date" name="fecha">
3   <input type="date" name="fecha2" value="2025-03-11">
4   <input type="datetime-local" name="fecha3">
5   <input type="time" name="hora">
6 </form>
```

<textarea>

- HTML ofrece el elemento *<textarea>* para insertar múltiples líneas de texto. Puede utilizar los atributos *rows* y *cols* para definir el alto y ancho del área en caracteres.

```
<form name="formulario" method="get" action="procesar.php">
  <div>
    <label>Texto:
      <textarea name="texto" cols="50" rows="6"></textarea>
    </label>
  </div>
</form>
```

<select>

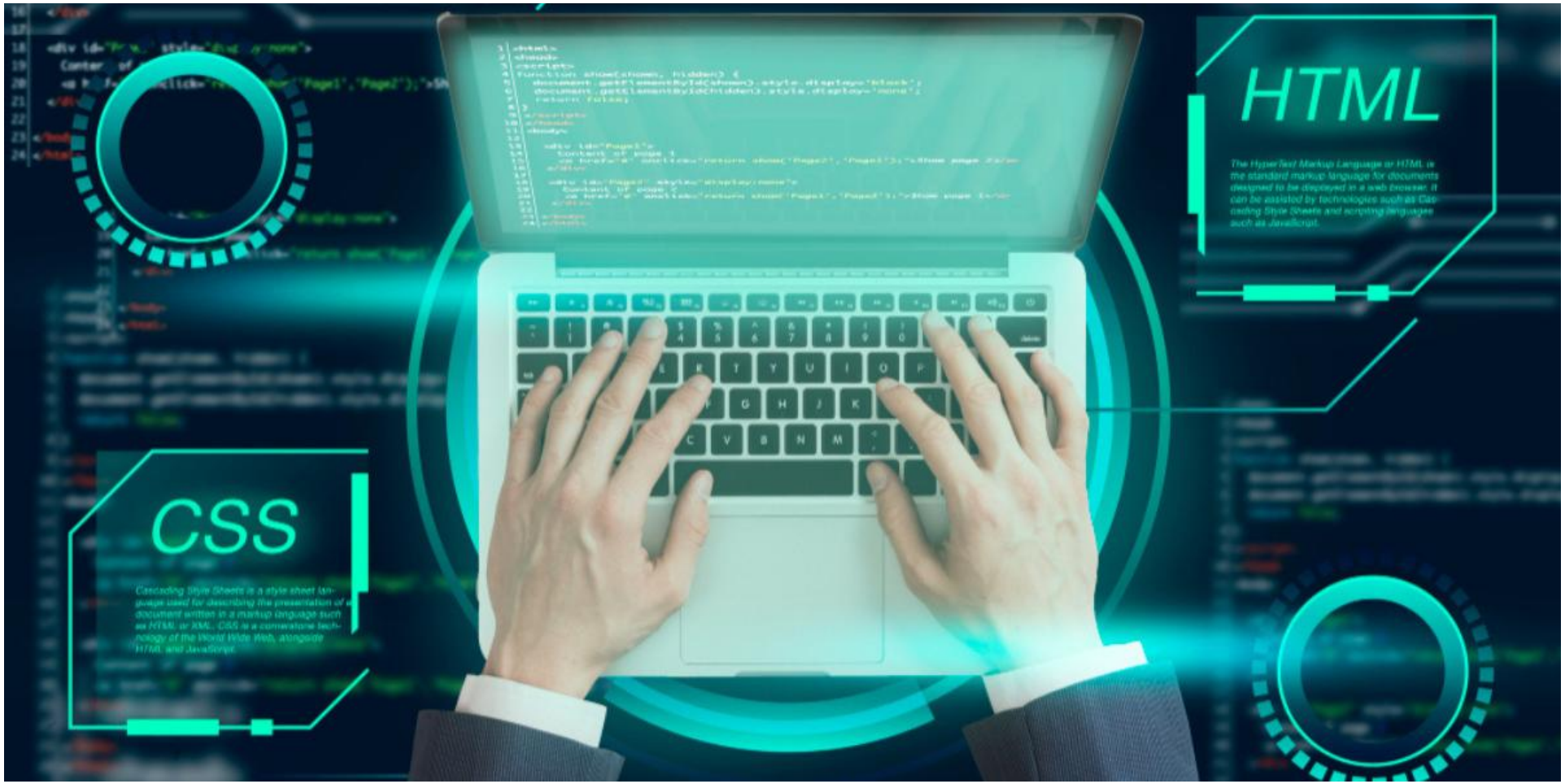
- Además de los tipos de campos de entrada *radio* y *checkbox* estudiados anteriormente, HTML ofrece el elemento *<select>* para presentar una lista de valores (o opciones) al usuario.
- El elemento *<select>* trabaja junto con el elemento *<option>* para definir las opciones. El elemento *<select>* debe incluir el atributo *name* para identificar el valor, y cada elemento *<option>* debe incluir el atributo *value* para definir el valor que representa.

```
1 <form name="formulario" method="get" action="procesar.php">
2   <div>
3     <label for="listado">Libros: </label>
4     <select name="libro" id="listado">
5       <option value="1">IT</option>
6       <option value="2">Carrie</option>
7       <option value="3">El Resplandor</option>
8       <option value="4">Misery</option>
9     </select>
10  </div>
11  <div>
12    <button type="submit">Enviar</button>
13  </div>
14 </form>
15
```

Atributos globales de elementos de formulario

- **disabled** → atributo booleano que desactiva el elemento.
- **readonly** → indica que el elemento no se puede modificar. Es de solo lectura
- **placeholder** → texto que se muestra en el fondo de un elemento
- **required** → Este es un atributo booleano que indica que el campo es obligatorio (o requerido)
- **autofocus** → El navegador mueve el foco al elemento tan pronto como se carga el documento
- **novalidate** → Atributo booleano para el elemento <form> que indica que el formulario no debería ser validado automáticamente por el navegador.

Por Qué HTML Sigue Siendo Fundamental en el Desarrollo Web Moderno



La Base Irremplazable de la Web

HTML es más crítico hoy que nunca, y entenderlo profundamente es lo que separa a un desarrollador junior de uno semi-senior o senior.

Cada aplicación web moderna, sin importar qué framework utilice, eventualmente se renderiza como HTML en el navegador. React, Vue, Angular: todos ellos son herramientas para generar HTML de manera más eficiente y mantenible. Un desarrollador que no comprende HTML a fondo está construyendo sobre cimientos que no entiende.

Accesibilidad: Un Imperativo Profesional

HTML semántico bien estructurado es la base de la accesibilidad web. Los lectores de pantalla, utilizados por personas con discapacidad visual, dependen completamente de la estructura semántica del HTML para navegar y comprender el contenido. Un *<button>* no es solo un elemento que hace clic; comunica al lector de pantalla que este elemento es interactivo. Un *<nav>* no es solo un contenedor; indica a las tecnologías asistivas que este es el menú de navegación principal.

SEO y Rendimiento Empresarial

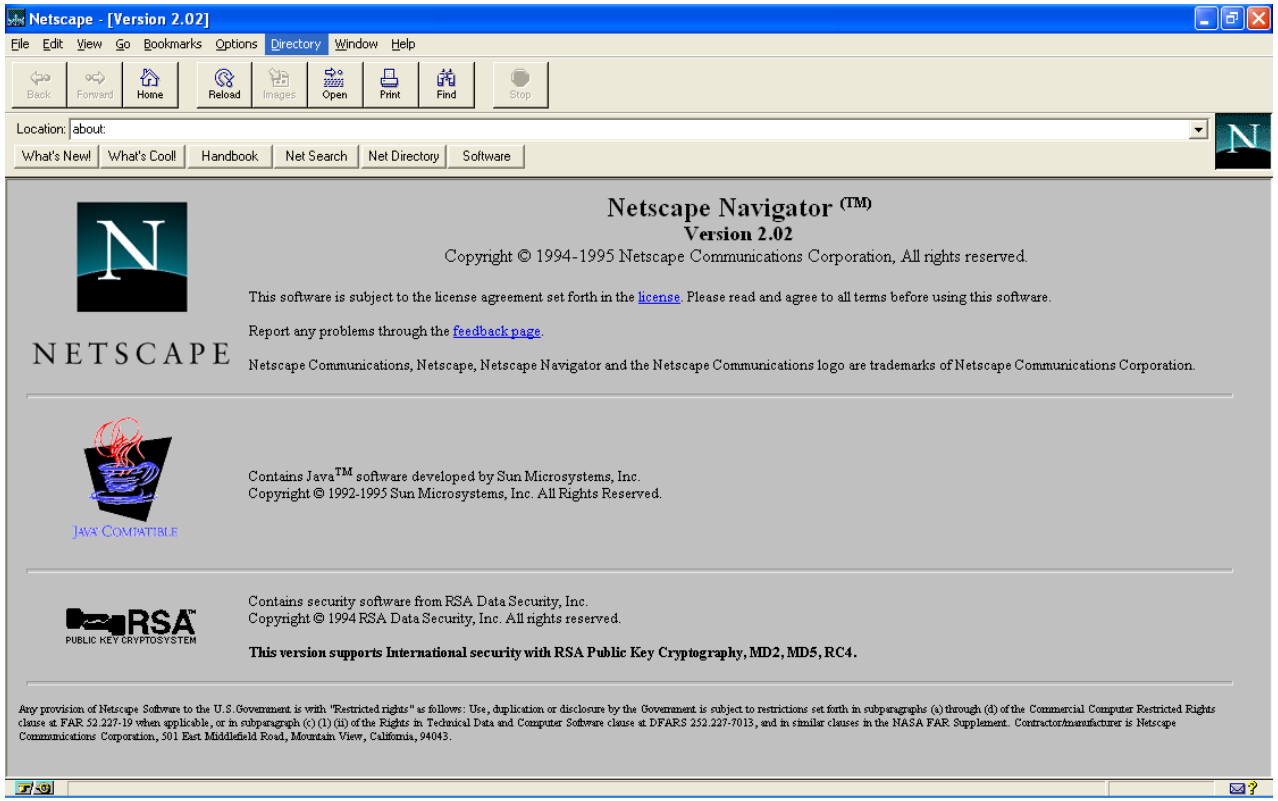
El HTML semántico tiene un impacto directo en el posicionamiento en motores de búsqueda. Google y otros buscadores utilizan la estructura HTML para entender el contenido de una página. Los encabezados correctamente jerarquizados *(<h1>* a *<h6>*), el uso apropiado de <article>, *<section>*, y otros elementos semánticos ayudan a los algoritmos de búsqueda a indexar el contenido correctamente.

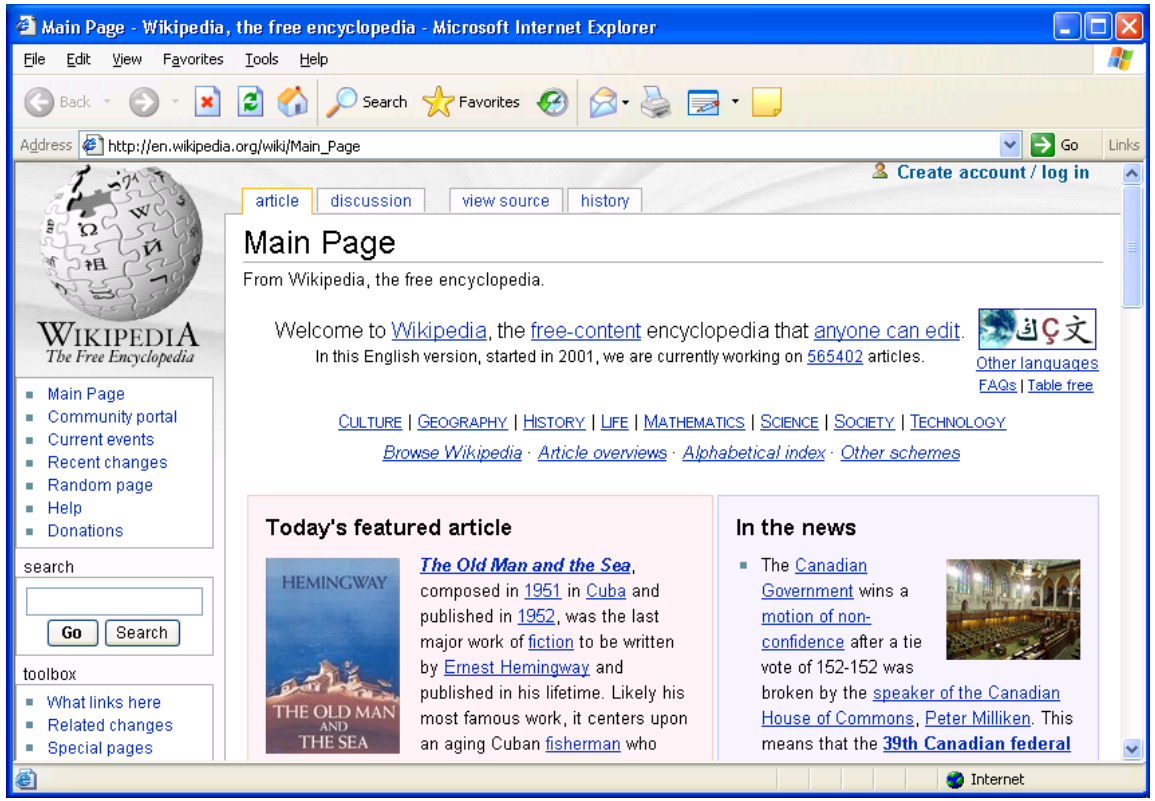
Un sitio web que no aparece en los resultados de búsqueda es un sitio que no genera negocio. Para empresas comerciales, el SEO puede ser la diferencia entre el éxito y el fracaso. Como desarrollador profesional, tu comprensión de HTML semántico impacta directamente el ROI (retorno de inversión) de los proyectos en los que trabajas.

Rendimiento y Velocidad de Carga

El HTML bien escrito también afecta el rendimiento. Los navegadores parsean y renderizan HTML de manera increíblemente eficiente, pero un HTML mal estructurado, excesivamente anidado, o innecesariamente complejo puede causar problemas de rendimiento, especialmente en dispositivos móviles.

Guerra de navegadores 1995-2001





W3C® WORLD WIDE WEB consortium

Contexto histórico

La Primera Guerra de los Navegadores fue el enfrentamiento entre:

- Netscape Navigator
- Internet Explorer (desarrollado por Microsoft)

En 1995, Netscape dominaba el mercado. Sin embargo, Microsoft lanzó Internet Explorer y lo integró gratuitamente dentro del sistema operativo Windows.

¿Qué implicó esto?

1. Microsoft aprovechó su posición dominante de Windows.
2. Internet Explorer pasó a estar instalado por defecto en millones de computadoras.
3. Netscape perdió rápidamente cuota de mercado.
4. Microsoft fue demandado por prácticas monopólicas en EE.UU.

¿Por qué esta guerra fue tan crítica?

Lo más importante no fue solo la competencia comercial, sino el impacto técnico:

- Cada navegador implementaba funcionalidades propietarias.
- No existía una adhesión estricta a estándares comunes.
- Los desarrolladores debían escribir código específico para cada navegador.

```javascript
if (navigator.appName === "Netscape") {
   // código específico
} else {
   // código para Internet Explorer
}
```

¿Por qué es importante mantener estándares?

Los estándares web son especificaciones técnicas definidas por organismos como el **World Wide Web Consortium (W3C).**

1. **Interoperabilidad**
   - Un mismo sitio debe funcionar igual en distintos navegadores.
   - Sin estándares → cada navegador interpreta el código a su manera.
2. **Portabilidad**
   - El código HTML, CSS y JS debe ser independiente del proveedor.
   - La web no pertenece a una empresa, sino que es un ecosistema abierto.
3. **Innovación sostenible**, cuando todos siguen reglas comunes:
   - Las mejoras son acumulativas.
   - No se rompe compatibilidad.
   - Se favorece el crecimiento del ecosistema.

4. Reducción de costos

Sin estándares:

- Más tiempo de desarrollo.
- Más testing.
- Más bugs.
- Mayor deuda técnica.

5. Estabilidad a largo plazo

- Compatibilidad futura.
- Evolución ordenada de tecnologías como HTML5 y CSS3.

Conclusiones

- HTML es un lenguaje de marcado que le dio inicio a la web
- Los elementos HTML se componen de etiquetas de apertura y cierre, contenido y atributos.
- Los formularios HTML nos permite obtener información del usuario.
- Los estándares son lo que hacen posible que la web funcione globalmente. Son un acuerdo técnico que protege el ecosistema. Son la base sobre la que se construyen frameworks, APIs y aplicaciones modernas.

En términos de ingeniería de software, los estándares garantizan consistencia, mantenibilidad y escalabilidad del ecosistema web.


UTN
FACULTAD REGIONAL
RAFAELA